Name: Kshitij Prabhu

SRN: PES1UG24CS914

Class: 5B

Rock Paper Scissors — Pygame

Lab no. 59

Repo: https://github.com/Kshitij-Prabhu/59_rock_paper_scissor

AI tool used: Claude

Before Video

https://drive.google.com/file/d/1bnQvpqXvQFAZPK6wQaOAPewU6WTHPBI6/view?usp=sharing

Shows the original buggy game. When the player picks ROCK and the CPU picks SCISSORS, the game incorrectly displays "You Lose" — demonstrating the inverted outcome bug.

After Video

https://drive.google.com/file/d/1XcZ0S9UjIaQ6Gix9KAUf45TaokwsYkJq/view?usp=sharing

Here you can see the game actually working the way it should. I picked Rock against Scissors and it finally said "You Win" — which sounds obvious, but that's exactly what the bug was breaking before. You can also see the score counting up toward 5, the game-over screen appearing when someone hits it, and pressing R brings everything back to zero.

Prompt Evidence / Chat History

https://claude.ai/share/00ce5293-d7d9-44b3-ad52-497063179981

The link above contains the complete AI-assisted session used throughout this lab — including bug analysis, fix implementation, and all three feature additions.

Bug Fixed — Inverted Win/Loss Outcomes

File: `game/game_engine.py` — method `determine_winner()`

The original rules dictionary had every outcome backwards. Fix: swap all six values so the correct player wins each combination.

```python
def determine_winner(self, player, cpu):
    """
    BUG: The original rules dict had the winners INVERTED.
    e.g. ("ROCK", "SCISSORS"): "CPU"  — but Rock BEATS Scissors,
    so the winner should be PLAYER, not CPU.
    FIX: Swap every "CPU" and "PLAYER" value in the rules dict.
    """
    if player == cpu:
        return "TIE"

    rules = {
        ("ROCK",     "SCISSORS"): "PLAYER",   # Rock beats Scissors
        ("SCISSORS", "PAPER"):    "PLAYER",   # Scissors beats Paper
        ("PAPER",    "ROCK"):     "PLAYER",   # Paper beats Rock
        ("SCISSORS", "ROCK"):     "CPU",      # Rock beats Scissors (CPU wins)
        ("PAPER",    "SCISSORS"): "CPU",      # Scissors beats Paper (CPU wins)
        ("ROCK",     "PAPER"):    "CPU",      # Paper beats Rock (CPU wins)
    }
    return rules.get((player, cpu), "TIE")
```

Feature 1 — First to 5 Wins

Added WIN_TARGET = 5. After each round the score is checked — if either side reaches 5, a dark overlay appears and the player can press R to restart.

```python
def play_round(self, choice):
    if self.match_over or self.showing_result:

    self.player_choice = choice

    # ── TASK 3: record player throw ──────────────────────
    self.player_history[choice] += 1

    # ── TASK 3: use adaptive CPU pick ────────────────────
    self.cpu_choice = self.cpu_pick()

    outcome = self.determine_winner(self.player_choice, self.cpu_choice)
    if outcome == "PLAYER":
        self.player_score += 1
        self.result_text  = f"You Win! {self.player_choice} beats {self.cpu_choice}."
        self.result_color = (80, 230, 120)
    elif outcome == "CPU":
        self.cpu_score   += 1
        self.result_text  = f"You Lose! {self.cpu_choice} beats {self.player_choice}."
        self.result_color = (240, 80, 80)
    else:
        self.result_text  = f"It's a Draw! Both picked {self.player_choice}."
        self.result_color = (240, 210, 80)

    # ── TASK 2: check for match winner ───────────────────
    if self.player_score >= WIN_TARGET:
        self.match_over        = True
        self.match_winner_text = f"YOU WIN THE MATCH! {WIN_TARGET}-{self.cpu_score}"
    elif self.cpu_score >= WIN_TARGET:
        self.match_over        = True
        self.match_winner_text = f"CPU WINS THE MATCH! {WIN_TARGET}-{self.player_score}"

    self.showing_result      = True
    self.round_resolved_time = pygame.time.get_ticks()
```

Feature 2 — Adaptive CPU

The CPU tracks the player's move history. After 4+ rounds, if any single move makes up more than 50% of throws, the CPU plays its counter 70% of the time.

```python
def cpu_pick(self):
    """
    After 4+ rounds, if one choice makes up >50% of the player's
    throws, bias the CPU toward the counter-move (70% of the time).
    Otherwise pick randomly.
    """
    total = sum(self.player_history.values())
    if total >= 4:
        most_common = max(self.player_history, key=self.player_history.get)
        if self.player_history[most_common] / total > 0.5:
            # 70% chance to play the counter, 30% random
            if random.random() < 0.7:
                return self.counter[most_common]
    return random.choice(self.choices)
```